OpenMP Memory Model, Flush, and Nowait

Lecture Notes for HPC Course — Parallel Regions (Part 2)

Lecture notes — HPC1 - Università di Trieste, A.Y. 2025-2026
Luca Tornatore, luca.tornatore@inaf.it

OpenMP Memory Model, Flush, and Nowait
- Lecture Notes for HPC Course — Parallel Regions (Part 2)
- 1. Why it exists a Memory Model?
  - The hardware reality
  - The software abstraction
- 2. Temporary View vs. Shared Memory
  - Example
  - The flush operation
- 3. Implicit Flush Points
  - Why this usually suffices
- 4. The cost of synchronization
- 5. The Flush Directive — A Rigorous Look
  - 5.1. Happens-before — the formal language
  - 5.2. What a flush actually does, formally
  - 5.3. The canonical producer-consumer with explicit flush
  - 5.4. The compiler fence is the most important thing
  - 5.5. What flush and atomic release actually do to the store buffer
- 6. Why the Flush Directive Is Hard to Use Correctly
- 7. Atomic operations with Memory-Ordering clauses
  - What atomic flushes
  - The producer-consumer pattern, modern version
  - Atomic vs. critical, briefly
  - What we're not covering here
- 8. The `nowait` Clause
  - The default: implicit barriers everywhere
  - Removing the barrier
  - When nowait is safe
  - When nowait helps
  - Example: independent loops
  - Example: dependent loops (nowait would be WRONG)
  - nowait with single
  - Contrast with master/masked
- 8. Putting It Together: The Memory Model in Practice
  - Trust implicit synchronization
  - Verify independence before using nowait
  - Reconsider if you think you need explicit flush
  - Atomics don't provide full fences
- 10. Why All This Matters for Performance
  - Barriers are expensive
  - False sharing in synchronization
  - Memory ordering affects optimization
- 11. Example Code
  - 10_memory_model.c

11_spinwait_flush.c
12_nowait.c
12. Summary Table
References

1. Why it exists a Memory Model?

Here we connect what we have learned learned in lectures about modern architectures and code optimization (cache hierarchy, coherency) and what we need for correct parallel programming.

The hardware reality

You know that in modern CPUs there is a strong memory hierarchy which is way more complex than a naive view of main memory. Instead:

- Each core has private L1 and L2 caches, and shares L3 cache (sometimes the L2, also).
- Writes may sit in store buffers before propagating outward
- The memory controller sees requests out of order
- Cache coherency protocols (MESI, MOESI) ensure *eventual* consistency, but "eventual" can mean hundreds or thousands of cycles

The consequence: when thread A writes `x = 5`, thread B may not see that value immediately, even though they share the same address space. Thread B might read a stale value from its own cache, or the write might still be sitting in thread A's store buffer.

The software abstraction

OpenMP runs on top of this hardware. It provides a *memory model* that addresses three questions:

1. **When** do writes become visible to other threads?
2. **What ordering guarantees** exist at synchronization points?
3. **How** can a programmer force visibility when needed?

The OpenMP memory model is, in effect, a contract between the programmer and the implementation. If you follow the rules, your program will behave correctly on any compliant implementation. If you violate the rules—say, by reading a shared variable without proper synchronization—the behavior is undefined, and you're at the mercy of whatever the hardware and compiler happen to do.

Remind: this is the same problem that cache coherency solves at the hardware level. The difference is that cache coherency operates automatically and transparently *once data are in the caches*; the OpenMP memory model requires the programmer to participate by using synchronization constructs correctly to enforce and ordering among ops and to deal correctly with what happens *before* the writes reach the L1 cache.

---

2. Temporary View vs. Shared Memory

OpenMP introduces a useful conceptual distinction:

- **Shared memory**: the "ground truth" that all threads eventually agree on. This is what you'd see if you could freeze the entire system and inspect main memory.

- **Temporary view**: what each thread currently "sees." This includes values in CPU registers, in store buffers, and in private cache lines that haven't yet been synchronized.

The key insight is that a thread's temporary view may be *stale*. It may contain values that another thread has already overwritten in shared memory, but that update hasn't propagated yet.

Example

Suppose thread 0 executes `x = 5` and thread 1 then reads `x`. Without synchronization:

- Thread 0's write may still be in its store buffer
- Even if it's in thread 0's L1 cache, thread 1's L1 cache may have a stale copy
- Thread 1 might read 0, or 5, or (in pathological cases on weakly-ordered architectures) something else entirely

This is not a bug in the hardware; it's the hardware doing exactly what it's designed to do. The bug is in the software, which failed to synchronize.

The flush operation

A **flush** is the mechanism that synchronizes a thread's temporary view with shared memory. Conceptually, a flush does two things:

1. **Pushes** all pending writes from the thread's temporary view to shared memory (makes the thread's writes visible to others)
2. **Pulls** fresh values from shared memory for subsequent reads (makes others' writes visible to this thread)

Think of it as: "push my writes out, pull fresh reads in."

Important subtlety: a flush is a *point-to-point* operation. It synchronizes THIS thread's view with shared memory. It does NOT wait for other threads to do anything. For communication to work, both sides must flush—one to push, one to pull.

---

3. Implicit Flush Points

In most OpenMP programs, you never need to think about flush explicitly, because OpenMP inserts **implicit flushes** at well-defined synchronization points:

Construct / Directive	Flush occurs...
`#pragma omp barrier`	At entry and exit
`#pragma omp critical`	At entry and exit
`#pragma omp parallel`	At entry of all threads; at exit (implicit barrier)
`#pragma omp for` (without `nowait`)	At exit (implicit barrier)
`#pragma omp single` (without `nowait`)	At exit (implicit barrier)
`#pragma omp sections` (without `nowait`)	At exit (implicit barrier)

Construct / Directive	Flush occurs...
`#pragma omp atomic`	Only for the specific memory location being updated
Lock operations	`omp_set_lock`, `omp_unset_lock`, `omp_test_lock`

Basically, every construct that involves synchronization includes a flush.

Why this usually suffices

Consider the typical pattern:

```
#pragma omp parallel
{
    #pragma omp for
    for (int i = 0; i < N; i++)
        a[i] = compute(i);

    // implicit barrier + flush here

    #pragma omp for
    for (int i = 0; i < N; i++)
        b[i] = a[i] * 2;    // reads a[]
}
```

The second loop reads values written by the first loop. Why does this work? Because the implicit barrier at the end of the first `for` includes a flush. By the time any thread enters the second loop, all writes to `a[]` have been pushed to shared memory, and all threads will see them.

This is why, in the vast majority of OpenMP programs, you never need explicit flush. The synchronization constructs handle it for you.

---

4. The cost of synchronization

The flip side of "trust the implicit synchronization" is that synchronization is not free. A flush — implicit or explicit — has a non-zero cost, with several components:

- **Compiler-side cost.** The compiler must conservatively assume that, across the flush, any shared variable might have been modified. This kills optimisations like register caching, hoisting loads out of loops, and reordering memory operations across the flush point. In code with many flushes, the compiler spends a lot of time being careful and the code that comes out of `-O3` is not what it could be in the absence of synchronization.
- **Hardware-side cost (architecture-dependent).** On x86-64 the cost is small, because the architecture is already strongly-ordered (TSO — Total Store Order: stores from a single core are seen in order by other cores, with the store buffer being the only source of laxity). On ARM, POWER and RISC-V the cost is higher, because the hardware *is* allowed to reorder memory operations more aggressively, and a flush has to translate into actual memory fences — `dmb ish` on ARM, `sync` on POWER, `fence` on RISC-V.
- **Latency cost when the flush actually waits.** If the store buffer is not empty when the flush is reached,

the core stalls until it drains. Drain time is usually a handful of cycles, but under heavy write pressure or contended cache lines it can balloon.

- **Synchronization cost on top of the flush.** A barrier, in particular, doesn't just flush — it also waits for all threads to arrive. On a many-thread, multi-socket system, the cost of a barrier scales (badly) with the number of threads and with how spread they are across NUMA nodes. The flush itself is the cheaper part.

In practice, for "well-shaped" parallel work — chunky parallel loops, occasional barriers, a few critical sections — none of this matters. The synchronization overhead is in the noise compared to the actual work.

It starts to matter when:

- the parallel granularity becomes small (many barriers, each guarding a short loop);
- you have many threads, especially across NUMA nodes;
- you have producer-consumer patterns where one thread is waiting and the others are signalling;
- you are in the inner loop of a numerical kernel and an OpenMP construct is sneaking a flush in there.

The lesson is twofold: don't sprinkle flushes around for "safety"; and, when synchronization shows up in a profile, look at whether you really need it where you have it. Sometimes the answer is "remove the barrier with `nowait`" (we'll see this in a few sections), sometimes the answer is "restructure the algorithm so that the dependency is broken or moved".

There are problems for which the bulk-parallel pattern of "do work, barrier, do next work, barrier" is not enough. Producer-consumer pipelines, lock-free data structures, work-stealing schedulers — all of these need finer-grained synchronization than barriers provide. The general way to build it is by hand, with explicit memory-visibility primitives. This is where the explicit `flush` directive comes in, and where, candidly, things start to be hard to get right. We'll only sketch the issues here in this introductory course; the full treatment belongs in the advanced course.

---

5. The Flush Directive — A Rigorous Look

The bare directive is:

```
#pragma omp flush               // strong flush of all shared variables
#pragma omp flush(x, y)         // flush of x and y
```

Since OpenMP 5.0, you can also specify a memory order:

```
#pragma omp flush acq_rel
#pragma omp flush release
#pragma omp flush acquire
#pragma omp flush seq_cst       // equivalent to the bare flush
```

To say what these *mean* properly we need a piece of formal language: the **happens-before relation**.

5.1. Happens-before — the formal language

The happens-before relation is the standard tool for reasoning about memory in concurrent systems. It was introduced by Lamport in 1978 and adopted, with variations, by all modern memory models — Java's, C11's, C++11's, OpenMP's. The core definition is:

> Operation **A happens-before** operation **B** if either:
>
> - **A** is sequenced before **B** in the same thread (ordinary single-thread program order), or
> - **A synchronizes-with B** (a cross-thread relation defined by the memory model), or
> - there exists an operation **C** such that A happens-before C and C happens-before B (transitivity).

The happens-before relation is a partial order. Two operations that are not related by happens-before are said to be *concurrent*, and the memory model gives no guarantee about the order in which they become visible.

The whole point of synchronization primitives is to introduce edges in the happens-before graph. A barrier introduces edges from every operation before the barrier in any thread, to every operation after the barrier in any other thread. A critical section introduces edges from one execution of the section to the next. And the flush operation can introduce such edges — *if used correctly*.

5.2. What a flush actually does, formally

A flush region with memory order *release* (or `acq_rel`, or `seq_cst`) is a **release flush**. A flush region with memory order *acquire* (or `acq_rel`, or `seq_cst`) is an **acquire flush**. The bare `#pragma omp flush` (or with `seq_cst`) is both: it acts simultaneously as a release flush and as an acquire flush — what the spec calls a **strong flush**.

The synchronizes-with relation in OpenMP (essentially OpenMP 5.0 §1.4.4, paraphrased) reads:

> A release flush by thread A **synchronizes-with** an acquire flush by thread B, if the two flushes share at least one variable in their flush sets, and B observes via memory a value that A's release flush made visible.

The chain from "A wrote some data" to "B sees that data" is then:

1. A writes `data` (and possibly `flag`).
2. A executes a *release flush* — this orders A's prior writes before subsequent operations and makes them visible.
3. B executes an *acquire flush* — this orders B's subsequent reads after the flush and pulls fresh values.
4. B reads `flag` (and `data`).

The release in step 2 *synchronizes-with* the acquire in step 3 if step 4 successfully observes the value that step 1 wrote. This synchronization edge, combined with single-thread program order on each side (step 1 sequenced before step 2; step 3 sequenced before step 4), gives a transitive happens-before edge from A's write of `data` to B's read of `data`. Hence B is guaranteed to see the updated `data`.

A subtle point about flush sets: with `flush(flag)` you only synchronize `flag`. The *ordering* effect, however, is broader than that — the spec says a release flush also orders all prior memory operations of the issuing thread, not just the listed ones. This matters in the producer-consumer pattern below: the release flush of `flag` orders the producer's prior write of `data`, even though `data` is not in the flush set. We rely on this — and it's something easy to get wrong.

5.3. The canonical producer-consumer with explicit flush

Consider the broken spin-wait we already looked at.

Consider a producer-consumer pattern:

```
int flag = 0;
int data = 0;

#pragma omp parallel sections
{
    #pragma omp section
    {
        // Producer
        data = compute_something();
        flag = 1;    // signal that data is ready
    }

    #pragma omp section
    {
        // Consumer
        while (flag == 0)
            ;        // spin until flag is set
        use(data);
    }
}
```

This code is broken:

- The producer writes `flag = 1`, but that write might sit in the producer's store buffer or cache
- The consumer reads `flag` in a tight loop, but it might keep reading from its own cache, seeing the stale value 0
- Result: infinite loop

With explicit flushes done properly, it becomes:

```
int data = 0, flag = 0;

#pragma omp parallel sections
{
    #pragma omp section
    {                                       // PRODUCER
        data = 42;
        #pragma omp flush(data)             // (a) push data
        flag = 1;
        #pragma omp flush(flag)             // (b) push flag, after data
    }

    #pragma omp section
    {                                       // CONSUMER
        int local_flag;
        do {
```

```
            #pragma omp flush(flag)            // (c) pull flag
            local_flag = flag;
        } while (local_flag == 0);
        #pragma omp flush(data)                // (d) pull data
        // now use data
    }
}
```

Why the flushes are arranged this way:

- **(a)** is the release flush for `data`. The producer's write `data = 42` is ordered before this flush.
- **(b)** is the release flush for `flag`. Note that it is *after* `flag = 1` and *after* (a). The order producer → `flush(data)` → `flag = 1` → `flush(flag)` means: when the consumer eventually observes `flag == 1`, `data == 42` is also already visible.
- **(c)** is the consumer's acquire flush for `flag`. Without it, the load `local_flag = flag` may use a stale value (or, in the `the_problem_v2.c` scenario, may be entirely hoisted out of the loop by the optimiser).
- **(d)** is the consumer's acquire flush for `data`. Once the consumer has decided that `flag == 1`, this flush ensures that the subsequent read of `data` sees the producer's value.

The pairing **(b)** ↔ **(c)** is the synchronizes-with edge that connects producer to consumer; **(a)** and **(d)** ensure that the data side respects the order. Remove any one of the four and the chain is broken.

This is implemented in `1_spinwait_flush.c`'s `spinwait_correct` function. The same file's `spinwait_forever` shows what happens when you forget the flushes and trust `volatile`. Spoiler: at `-O3`, it hangs.

5.4. The compiler fence is the most important thing

Here is a point that the slide deck (the "with/without flush" picture) gets across in a roundabout way and that deserves to be stated cleanly: **on x86-64, the dominant role of a flush is the compiler fence, not the hardware fence**.

The reason is that x86-64 has Total Store Order: stores from a single core are seen in program order by all other cores; the only reordering allowed in hardware is store-load (a younger load to a different location overtaking an older store still in the buffer). For the producer-consumer pattern above, TSO gives us exactly the ordering we need at the hardware level, *for free*. The store buffer drains automatically into L1, in a few cycles; coherency takes care of cross-core visibility, in a few tens of cycles. None of that requires a fence.

But none of that helps if the compiler optimised the consumer's load of `flag` out of the loop, because then there is *no load instruction for the hardware to execute*. The compiler fence in `#pragma omp flush(flag)` is what forces the compiler to actually issue a load each time around the loop, fetching `flag` from memory rather than re-reading a stale register copy.

On ARM, POWER and RISC-V, the hardware-fence side of `flush` is genuinely needed too, because those architectures *do* allow store-store and load-load reordering in hardware. But on x86-64 the ratio of "compiler fence matters" to "hardware fence matters" is, in practice, lopsided in favour of the compiler. The detailed mechanics — what the store buffer does, why `release` is cheap on x86 but not on ARM, and what `seq_cst` actually costs — are in §5.5.

Remind: this is also why `volatile` in C is *not* a substitute. `volatile` forces the compiler to emit each load and each store, but it gives no ordering guarantees between accesses to different `volatile` variables (let alone between `volatile` and non-`volatile`), and it does no hardware fence at all. It is enough to defeat the `the_problem_v2` hang at `-O3`, but not enough for any correct cross-thread synchronization.

5.5. What flush and atomic release actually do to the store buffer

A confusion that the name *flush* invites, and that is worth addressing head-on: a flush does **not** push dirty cache lines back to DRAM. It does not need to.

On a cache-coherent system, a write that has reached the L1 cache in *Modified* state is already visible to any other core that asks for that line — the coherency protocol (MESI, MOESI, MESIF) transfers the line cache-to-cache, DRAM never involved. Eventually the line will be written back, but that's a function of cache eviction policy and is irrelevant for correctness.

The second confusion, subtler and worth being precise about, is the relationship between flush and the store buffer. Let's be careful here.

Stores are *not* "stuck" in the store buffer — they are only *delayed*. The buffer is a finite FIFO; stores commit to the L1 cache autonomously and continuously as the cache lines they target become available in the right MESI state. The drain happens whether or not you issue a fence. So flush doesn't *cause* the drain. What a hardware fence (`MFENCE`, or a `LOCK`-prefixed instruction on x86) does is cause the *core* to stop retiring subsequent instructions until the drain has finished. The drain was already going to happen; the fence just forces the CPU to *wait for it* before proceeding.

That "wait for the drain" is where the flush cost actually comes from on the hardware side. If the store buffer happened to be empty when the fence arrived, it costs essentially nothing. If the producer just finished writing a large array and immediately fenced, there may be tens of cycles of stall while the remaining stores drain.

This brings us to a practically important distinction between memory-order strengths on x86-64:

- **`release` write** (e.g. `#pragma omp atomic write release`): TSO already guarantees store-store ordering — a core's stores are seen by other cores in program order. So on x86 there is nothing for a release fence to enforce at the hardware level. The `release` atomic write typically compiles to a plain `mov` plus a *compiler fence* only. The core never stalls on a drain.
- **`seq_cst` write** (e.g. bare `#pragma omp flush` or `#pragma omp atomic write seq_cst`): the only reordering TSO *permits* is store-load (a younger load to a different address overtaking an older buffered store). Sequential consistency must prevent even that. So the runtime emits `MFENCE` or a `LOCK`-prefixed instruction, and the core *does* stall until the store buffer is empty.

The practical cost hierarchy on x86-64 is roughly:

```
relaxed write    →  mov only, no fence, compiler may reorder freely
release write    →  mov + compiler fence, no hardware stall
seq_cst write    →  MFENCE / LOCK + compiler fence, hardware stalls for drain
```

On **ARM, POWER and RISC-V** the picture is different: those architectures permit store-store reordering in hardware, so a `release` write *must* emit a hardware fence (ARM `dmb ishst`, POWER `lwsync`, RISC-V `fence w,w`). The release is not free there. This is one of the reasons why code that looks and runs correctly on x86 can behave differently on ARM — if you relied on the TSO store-store guarantee without encoding it explicitly, you get a data race on weaker architectures.

The bottom line: `flush` and `atomic release/seq_cst` enforce **ordering** — they constrain *when* operations become visible to other threads, not *where* the data physically resides. The store-buffer drain is the underlying hardware mechanism on the write side; the fence is a synchronization point against that mechanism. Coherency then propagates the committed cache lines to other cores automatically, without any explicit push.

This also explains why, as argued in §5.4, **the compiler fence is the dominant effect on x86-64 in most OpenMP programs**. If you use `release`/`acquire` atomics (the recommended modern pattern from §7), the hardware cost on x86 is near zero; the only real cost is the compiler barrier that prevents hoisting and reordering. The hardware-fence stall is reserved for `seq_cst`, which you should use sparingly in hot paths.

---

6. Why the Flush Directive Is Hard to Use Correctly

If you've read §5 carefully you may already suspect the punchline of this section: the `flush` directive is a low-level primitive with semantics that are subtle in three different directions at once — flush set, memory order, and pairing — and getting all three right at the same time is harder than it looks. Here are the typical mistakes.

(a) Forgetting that a flush is one-sided. The producer flushes after writing; the consumer also has to flush before reading. A flush only by the producer means the writes are pushed to memory but the consumer's view is never refreshed; the consumer keeps using its stale (possibly register-held) copy. A flush only by the consumer means the consumer pulls fresh values, but if the producer never pushed, there is nothing fresh to pull. *Both* sides must participate.

(b) Getting the order wrong. In the producer, you need `data = 42; flush; flag = 1; flush(flag)`. If you swap the writes — `flag = 1; ... ; data = 42` — you produce a window during which the consumer can observe `flag == 1` while `data` is still 0. Same disaster if you put the flush of `data` *after* the write of `flag`. The flush imposes an ordering, but only with respect to operations *before* it; operations *after* the flush can still be reordered relative to the next thing.

(c) Misusing `flush(list)`. When you list variables, only those variables are part of the flush set, and only their values are guaranteed to be synchronized in the strict sense. The ordering effect of a release flush still extends to *all* prior writes by that thread — but if you rely on this, you're relying on a slightly subtle reading of the spec, and it's easy to convince yourself you're being clever when you're being incorrect. The safe rule of thumb: if the variables on which you depend are not in the flush set, use a bare `#pragma omp flush` (which flushes everything). The cost is higher, but the semantics are unambiguous.

(d) Conflating flush with synchronization. A flush is *not* a barrier and *not* a wait. Two threads each calling `#pragma omp flush` are doing two unrelated local operations; they don't meet, they don't signal. If what you actually want is "thread A waits for thread B to finish work", you want a barrier, or task dependencies, or a lock — not a flush.

(e) The historical baggage of the name. As we already discussed, "flush" suggests "writeback to DRAM", which is not what flush does. It is closer in spirit to a C11 `atomic_thread_fence` than to anything resembling a write-back. The terminology is unfortunate, and the OpenMP committee has, in 5.0, layered the explicit memory-order clauses on top precisely because the unadorned name was misleading.

(f) The reasoning is hard. Reconstructing a correct happens-before argument from a sprinkling of `flush` directives is, frankly, error-prone even for experienced practitioners. There are entire papers in the academic literature about subtle bugs in OpenMP runtimes and applications that come from incorrect flush usage. If you find yourself writing flush-heavy code, it is almost always a sign that you should be using a higher-level primitive instead.

The practical advice that comes out of this:

1. **Trust implicit synchronization** wherever you can. Barriers, critical sections, atomics, locks, and worksharing constructs already insert flushes at the right places.
2. **If you remove a barrier with `nowait`, verify the independence** of what comes after. If the next computation reads what the current one wrote, the barrier is what makes the read correct.
3. **If you think you need an explicit `flush`, reconsider.** Can you use a barrier? A critical? An atomic? A task with `depend`? In 95% of cases, one of these is clearer and equally efficient.
4. **If you really need a flush** — say, in a custom spin-wait or a hand-rolled lock — strongly prefer the OpenMP 5.0+ atomic operations with explicit memory orders (see §7 below). They give the same ordering guarantees with a syntax that makes the intent explicit, and the compiler is much better equipped to optimize them sensibly.

I don't want to leave the impression that explicit `flush` should never be used. There are legitimate use cases — busy-wait synchronization, lock-free data structures, custom barrier implementations. The point is that those are the use cases of *advanced* parallel programming, and even there the modern way is to write them with atomics rather than with bare flushes.

---

7. Atomic operations with Memory-Ordering clauses

OpenMP 5.0 (and finer-grained controls in 5.1) gives you atomic operations parameterized by a memory order. The syntax is:

```
#pragma omp atomic [memory-order] read
   v = x;

#pragma omp atomic [memory-order] write
   x = expr;

#pragma omp atomic [memory-order] update          // the default kind
   x++;                                           // or x op= expr, etc.

#pragma omp atomic [memory-order] capture
   { v = x; x = expr; }                           // various forms
```

with `memory-order` being one of `seq_cst`, `acq_rel`, `release`, `acquire`, `relaxed`. The default is `seq_cst` for backward compatibility.

These map more or less directly onto the C11/C++11 memory ordering model. For most purposes, the default `seq_cst` is fine — it gives the strongest guarantees, and on x86-64 the cost difference between `seq_cst` and `acq_rel` is small. If you need explicit control over memory ordering for a hot path, you're in expert territory and you should think carefully — but at that point you also know what `relaxed` means and when `release`/`acquire` are enough.

What atomic flushes

The point that matters here, and was already mentioned in §3: `atomic` flushes **only the specific memory location being updated** — not the whole shared state. This is both a feature (efficiency: it's much cheaper than a full flush) and a pitfall (limited visibility guarantees: other variables around the atomic are not synchronized by it).

The producer-consumer pattern, modern version

The producer-consumer pattern with atomics is, in my opinion, much cleaner than with explicit flushes:

```
int data = 0, flag = 0;

// PRODUCER
data = 42;
#pragma omp atomic write release
flag = 1;

// CONSUMER
int local_flag;
do {
    #pragma omp atomic read acquire
    local_flag = flag;
} while (local_flag == 0);
// now read data — guaranteed to see 42
```

The `release` on the producer's write of `flag` is a release fence: all prior writes by the producer (including `data = 42`) are ordered before the atomic store. The `acquire` on the consumer's read of `flag` is an acquire fence: all subsequent reads (including the read of `data`) are ordered after the atomic load. The synchronizes-with edge is set up by the very fact that the consumer reads (eventually) the value the producer wrote, and the happens-before chain runs

`data = 42` → `flag = 1 (release)` → `local_flag = 1 (acquire)` → `read data`.

Done. No explicit `flush` directives, no flush-set subtleties, no order-of-operations gotchas around the flushes themselves. The semantics are explicit in the directive: "this is a release, this is an acquire", and the compiler can lower it to the right hardware fence (or no fence at all, on x86-64 for read/write of word-size aligned data) and the right compiler-fence behaviour.

This example is in `1_spinwait_flush.c`'s `spinwait_with_atomic` function. Note the explicit comment in the source:

> IN PRACTICE DO NOT USE THE FLUSH SEMANTICS, SINCE IT IS MORE A MEMORY FENCE THAN A PROPER SYNCHRONIZATION (ALTHOUGH THE FLUSH(LIST) IS IN PRINCIPLE EQUIVALENT TO ATOMICS). RECONSTRUCTING THE HAPPENS-BEFORE PATTERN IS VERY ERROR-PRONE.

That comment is the operational summary of all of §6.

Atomic vs. critical, briefly

A common question: when to use `atomic` vs. `critical`?

- **Atomic** uses hardware primitives (locked instructions, compare-and-swap). It's efficient for simple operations on a single memory location.
- **Critical** uses a lock/mutex. It can protect arbitrary code, not just single statements.

For incrementing a shared counter, `atomic` is typically much faster. For protecting a complex update involving multiple variables, you need `critical` (or explicit locks). One gotcha worth noting: **atomic and critical do not interlock with each other.** If one thread updates `x` with `atomic` and another updates the same `x` from inside a `critical` region, you have a race. Pick one mechanism per shared object and use it consistently.

What we're not covering here

A full treatment of atomic memory orders, of `compare-and-swap` / `atomic capture compare`, of relaxed-ordering algorithms, and of the lock-free patterns built on top, belongs in the advanced course. For this introductory course, three things to remember:

1. `#pragma omp atomic write release` is the modern way to publish a flag from a producer.
2. `#pragma omp atomic read acquire` is the modern way for a consumer to observe it.
3. If you ever feel tempted to write a bare `#pragma omp flush(...)` in real code, ask yourself first whether one of the two above does what you want. Almost always it will.

---

8. The `nowait` Clause

Now we shift gears to a different topic: the `nowait` clause, which removes implicit barriers from worksharing constructs.

The default: implicit barriers everywhere

By default, worksharing constructs (`for`, `sections`, `single`) have an implicit barrier at their end. All threads must finish the construct before any thread can proceed.

```
#pragma omp parallel
{
    #pragma omp for          // implicit barrier at end
    for (int i = 0; i < N; i++)
        a[i] = compute(i);

    // <-- all threads wait here

    #pragma omp for
    for (int i = 0; i < N; i++)
        b[i] = other(a[i]);
}
```

This is safe but potentially wasteful. If some threads finish the first loop early, they sit idle while waiting for slower threads.

Removing the barrier

The `nowait` clause removes the implicit barrier:

```
#pragma omp parallel
{
    #pragma omp for nowait   // NO barrier
    for (int i = 0; i < N; i++)
        a[i] = compute(i);

    // threads proceed immediately

    #pragma omp for
    for (int i = 0; i < N; i++)
        b[i] = other(i);     // does NOT read a[]
}
```

Now threads that finish the first loop can start the second loop immediately, while slower threads are still working on the first.

When nowait is safe

The rule is simple: **you can use nowait when the next work does NOT depend on results from the current worksharing construct.**

Safe cases:

- Two loops operating on completely independent arrays
- A `single` block that does logging or I/O that doesn't affect subsequent computation
- Multiple independent computations that happen to be expressed as sequential loops

Unsafe cases:

- Second loop reads data written by first loop
- Any dependency between the constructs

When nowait helps

Nowait is most beneficial when:

1. **Load is imbalanced.** If some threads have more work than others, nowait lets fast threads start new work instead of waiting for slow threads.
2. **There are many sequential worksharing constructs.** Each barrier has overhead (synchronization, cache traffic). Removing unnecessary barriers reduces this overhead.
3. **The parallel region has a final barrier anyway.** Remember that `#pragma omp parallel` has an implicit barrier at the end. If you use nowait on intermediate constructs, the final barrier still ensures everything is finished before the region ends.

Example: independent loops

```
#pragma omp parallel
{
    #pragma omp for nowait
    for (int i = 0; i < N; i++)
        a[i] = sin((double)i);

    #pragma omp for nowait
    for (int i = 0; i < M; i++)
        b[i] = cos((double)i);

    #pragma omp for nowait
    for (int i = 0; i < K; i++)
        c[i] = tan((double)i * 0.001);

}  // implicit barrier here
```

All three loops are independent, so nowait is safe on all of them. The barrier at the end of the parallel region ensures everything is done before we proceed.

Example: dependent loops (nowait would be WRONG)

```
#pragma omp parallel
{
    #pragma omp for nowait   // WRONG!
    for (int i = 0; i < N; i++)
        a[i] = compute(i);

    #pragma omp for
    for (int i = 0; i < N; i++)
        b[i] = a[i] * 2;     // reads a[]
}
```

The second loop reads `a[i]`. With nowait on the first loop, a thread might start reading `a[i]` before another thread has finished writing it. Race condition.

nowait with single

The `single` construct normally has an implicit barrier: other threads wait while one thread executes the block.

```
#pragma omp parallel
{
    #pragma omp single       // other threads wait
    {
        printf("Starting...\n");
    }
    // all threads proceed together

    #pragma omp for
    for (int i = 0; i < N; i++)
        work(i);
}
```

With `nowait`, other threads don't wait:

```
#pragma omp parallel
{
    #pragma omp single nowait  // other threads proceed immediately
    {
        printf("Starting...\n");
    }
    // other threads may already be in the for loop

    #pragma omp for
    for (int i = 0; i < N; i++)
        work(i);
}
```

This is useful when the single block is independent of the subsequent work—logging, progress reporting, or initialization that isn't needed by the next computation.

Contrast with master/masked

The `master` construct (deprecated in OpenMP 5.1, replaced by `masked`) has NO implicit barrier by design. Use it when you need specifically thread 0 to do something while others proceed.

```
#pragma omp parallel
{
    #pragma omp master      // no barrier by design
    {
        // only thread 0 does this; others proceed
    }

    // all threads (including 0) proceed here
    #pragma omp for
    ...
}
```

The difference: `single` picks an arbitrary thread (implementation choice); `master`/`masked` always picks thread 0 (or, with `masked`, a specified thread).

8. Putting It Together: The Memory Model in Practice

Let me summarize the practical guidance:

Trust implicit synchronization

The implicit flushes at barriers, critical sections, and lock operations handle the vast majority of cases. If you're using these constructs correctly, you don't need to think about explicit flush.

Verify independence before using nowait

Before adding nowait to a worksharing construct, ask: "Does the next work read anything that this construct writes?" If yes, you need the barrier. If no, nowait is safe.

If in doubt, keep the barrier. The cost of an unnecessary barrier is typically small (microseconds). The cost of a race condition is debugging hell.

Reconsider if you think you need explicit flush

If you find yourself writing `#pragma omp flush`, step back. Are you implementing your own synchronization? Can you use barrier, cr2itical, atomic, or task dependencies instead?

Explicit flush is correct when used properly, but it's easy to get wrong. The failure mode is subtle—the program might work on your machine, in your test cases, most of the time. Then it fails in production, or on a different architecture, or when the moon is in the wrong phase. Race conditions are debugging nightmares.

Atomics don't provide full fences

Remember that `#pragma omp atomic` flushes only the target variable. If you're using atomics for signaling between threads (like the flag in a producer-consumer), you may need additional synchronization for other shared data. This is where the memory ordering clauses (`seq_cst`, `acq_rel`, etc.) become relevant.

---

10. Why All This Matters for Performance

This is where we circle back to the HPC motivation.

Barriers are expensive

A barrier involves:

1. Cache traffic: each thread's writes must propagate to shared memory
2. Synchronization overhead: some mechanism (typically atomic counters or flags) to detect when all threads have arrived
3. Potential memory bandwidth pressure: if all threads flush at once, there's a burst of coherency traffic

On a 64-core machine, a barrier might cost tens of microseconds. That's negligible for coarse-grained parallelism, but it adds up for fine-grained loops.

False sharing in synchronization

The synchronization mechanisms themselves can suffer from false sharing. If the runtime uses an array of per-thread flags for barrier implementation, and those flags are on the same cache line, you get ping-ponging. Good OpenMP implementations pad these structures, but it's worth being aware of.

Memory ordering affects optimization

The memory model constrains what the compiler can do. Within a thread, the compiler can reorder loads and stores for efficiency—as long as the result is the same *for that thread*. But it cannot reorder across flush points, because that could change what other threads see.

This means that adding synchronization doesn't just cost the synchronization overhead; it also inhibits compiler optimization of surrounding code. One more reason to use synchronization judiciously.

---

11. Example Code

Three example files accompany these notes:

10_memory_model.c

Demonstrates visibility with different synchronization mechanisms:

- Barrier with implicit flush
- Critical section
- Atomic operations
- Broken version without synchronization (intentional race condition)
- Explicit flush behavior
- Implicit flush at parallel region boundaries

11_spinwait_flush.c

Shows the one case where explicit flush is truly necessary—busy-wait synchronization:

- Correct producer-consumer with explicit flush
- Alternative using atomic operations (preferred in practice)
- Multi-producer example
- Explanation of why `volatile` is not sufficient

The volatile point is worth emphasizing: students with C background often think `volatile` solves visibility problems. It doesn't. `volatile` tells the compiler not to optimize away memory accesses, but it provides no memory ordering guarantees and no cache synchronization. In OpenMP, use flush or synchronization constructs, not volatile.

12_nowait.c

Demonstrates correct and incorrect use of nowait:

- Independent loops (safe to use nowait)
- Dependent loops (nowait would cause race)
- single with nowait
- Chained independent loops
- Partial dependencies (selective use of nowait)
- Load imbalance scenario (where nowait helps most)

---

12. Summary Table

Concept	What it means	When to use
Temporary view	Each thread may have stale copies of shared variables in registers/cache	Understand why synchronization is needed
Implicit flush	Happens automatically at barrier, critical, parallel entry/exit, lock ops	Most common case—rely on this
Explicit flush	`#pragma omp flush`—forces sync of thread's view with shared memory	Rare: spin-locks, lock-free code
nowait	Removes implicit barrier from for/sections/single	When next work is independent
barrier	`#pragma omp barrier`—all threads sync here	When you need ALL threads to finish before proceeding

---

References

- OpenMP 5.2 Specification, Chapter 1.4 (Memory Model): The authoritative source, but dense. Sections 1.4.4 (The Flush Operation) and 1.4.6 (OpenMP Memory Consistency) are the most relevant.
- OpenMP 5.2 Examples document (Section 6): Contains worked examples of memory model behavior, including explicit flush usage.
- Mattson, Sanders & Massingill, *Patterns for Parallel Programming*: Good discussion of synchronization patterns at a higher level of abstraction.
- Boehm & Adve, "Foundations of the C++ Concurrency Memory Model" (PLDI 2008): The theoretical foundations of weak memory models. Heavy reading, but useful if you want to understand *why* the rules are what they are.

Notes version: March 2026. These notes accompany the slides on OpenMP Memory Model (openmp_memory_model.pptx) and example codes 10_memory_model.c, 11_spinwait_flush.c, 12_nowait.c.